1. The Big Picture

At its core, this project asks a question that sits squarely in AAE 568: *given a spacecraft whose motion is governed by a known set of differential equations, find the control history $u(t)$ that steers it from an initial state to a desired final state while minimizing a cost functional.* That is the definition of an optimal control problem, and the theory we use to solve it—Hamiltonian mechanics, Pontryagin's Minimum Principle, the two-point boundary value problem, and numerical shooting—comes directly from the course.

What makes the project more than a homework exercise is the *setting*: the dynamics are the nonlinear, three-dimensional Circular Restricted Three-Body Problem (CR3BP), the boundary conditions are a 185 km LEO and a 9:2 near-rectilinear halo orbit around Earth–Moon L_2, and the cost functionals span minimum energy, minimum time, and minimum fuel. The nonlinearity and high dimensionality of this problem force us to go beyond the analytical examples worked in class and develop numerical infrastructure—direct collocation, mesh refinement, homotopy continuation—that *extends* the course material. But every extension rests on a foundation that was taught in the lectures.

This document tells that story in three acts: the theory that comes straight from the course (§2–3), the numerical method the course teaches for solving the resulting problem (§4), and the techniques we built on top of that foundation when the course's tools alone were not enough (§5–6).

2. The Theory—Directly from AAE 568

The theoretical backbone of our project is the material from the first half of the semester: static optimization, calculus of variations, and the Minimum Principle. We did not modify or extend this theory—we *applied* it, in the same way the course applies it to the chemical reactor and double-integrator examples, just on harder dynamics.

2.1 Formulating the Bolza Problem (Lecture Notes, pp. 141–142)

The course defines the general optimal control problem in Bolza form:

$$\min\ J = \phi\big(x(t_f),\, t_f\big) + \int_{t_0}^{t_f} \mathcal{L}\big(x,\, u,\, t\big)\, dt, \qquad \text{s.t. } \dot{x} = f(x, u, t),\ \ x(t_0) = x_0.$$

Our project is a direct instantiation of this template. The state is $x = [x, y, z, v_x, v_y, v_z]^\top$ (position and velocity in the rotating frame), the control is $u = [u_x, u_y, u_z]^\top$ (low-thrust acceleration), and f is the CR3BP dynamics augmented by the control. We set $\phi = 0$ and work with three different running costs:

- **Minimum energy:** $\mathcal{L} = \|u\|^2$, which is the quadratic cost from lecture (p. 142) with $Q = 0$, $R = I$.
- **Minimum time:** $\mathcal{L} = 1$, so $J = t_f - t_0$, with t_f free.
- **Minimum fuel:** $\mathcal{L} = \|u\|$, the L^1 norm that promotes sparsity in the control.

There is nothing novel in this formulation step—it is the direct application of the framework from the notes. The novelty is only in the choice of f, which happens to be a 6D nonlinear system with two gravitational singularities rather than the 2D examples in the lectures.

2.2 The Hamiltonian and Euler–Lagrange Equations (pp. 143–147)

Following the calculus of variations development in class, we adjoin the dynamics constraint with a costate $\lambda(t)$ and define the Hamiltonian:

$$\mathcal{H} = \mathcal{L}(x, u) + \lambda^\top f(x, u).$$

Setting $\delta J_a = 0$ produces the three Euler–Lagrange necessary conditions. For our minimum-energy CR3BP problem, these become:

(a) Costate equation $\dot{\lambda} = -(\partial\mathcal{H}/\partial x)^\top$:

$$\dot{\lambda}_r = -\mathbf{U}_{xx}\,\lambda_v, \qquad \dot{\lambda}_{v_x} = -\lambda_{r_x} + 2\lambda_{v_y}, \qquad \dot{\lambda}_{v_y} = -\lambda_{r_y} - 2\lambda_{v_x}, \qquad \dot{\lambda}_{v_z} = -\lambda_{r_z},$$

where $\mathbf{U}_{xx}$ is the 3×3 Hessian of the CR3BP pseudo-potential and the ± 2 terms come from the Coriolis acceleration. This is the same $\dot{\lambda} = -(\partial f/\partial x)^\top \lambda$ structure from the lectures, just evaluated for a more complex f.

(b) Transversality $\lambda(t_f) = (\partial\phi/\partial x)^\top|_{t_f}$: since $\phi = 0$ and the final state is fixed, the terminal costates are free (they are what we solve for).

(c) Stationarity $\partial\mathcal{H}/\partial u = 0$: for the quadratic running cost,

$$2u + \lambda_v = 0 \quad \Longrightarrow \quad u^* = -\tfrac{1}{2}\,\lambda_v.$$

This is the optimal control law. It appears verbatim in our code as `u = -0.5 * lam_v` (line 47 for two-body, line 179 for CR3BP in `dynamics.py`). The form is identical to the "linear state feedback" result that Kalman's LQR produces for the quadratic cost—our system is not linear, but the optimal control still depends linearly on the costate because the cost is quadratic in u.

2.3 Pontryagin's Minimum Principle (pp. 153–160)

When we impose a thrust bound $\|u\| \le u_{\max}$, the unconstrained stationarity condition no longer applies. The course replaces it with PMP:

$$\mathcal{H}(x^*, u^*, \lambda^*, t) \;\le\; \mathcal{H}(x^*, u, \lambda^*, t) \qquad \forall\, u \in U.$$

For the minimum-time problem ($\mathcal{L} = 1$), minimizing $\mathcal{H} = 1 + \lambda^\top f$ over the admissible set gives the bang-bang control law derived in the lectures (pp. 170–180):

$$u^*(t) = -u_{\max}\,\frac{\lambda_v}{\|\lambda_v\|}.$$

The thrust is always at maximum magnitude, pointing opposite to the velocity costate. This is implemented directly in `dynamics.py` (lines 62–89). The switching function, the bang-bang structure, the relationship between λ_v and the optimal thrust direction—all of this is textbook AAE 568.

2.4 Transversality for Free Final Time (pp. 150–152)

For the minimum-time problem, t_f is not given. The course derives the additional necessary condition:

$$\left(\frac{\partial\phi}{\partial t} + \mathcal{H}\right)\bigg|_{t_f} = 0.$$

Since $\phi = 0$ and the system is autonomous, this reduces to $\mathcal{H}(t_f) = 0$. In our shooting code (`shooting.py`, lines 58–107), t_f is an additional unknown in the shooting vector, and $\mathcal{H}_f = 0$ is an additional residual equation—a direct implementation of this transversality condition.

3. Sufficient Conditions (p. 152)

The course also covers when the necessary conditions are actually sufficient. The Legendre–Clebsch condition requires:

$$\frac{\partial^2 \mathcal{H}}{\partial u^2} > 0 \qquad \forall\, t \in [t_0, t_f].$$

For our minimum-energy formulation, $\mathcal{H} = \|u\|^2 + \lambda_v^\top u + \ldots$, so $\mathcal{H}_{uu} = 2I$, which is positive definite everywhere. This confirms that the extremals found by our shooting method are indeed local minima, not saddle points. This check is a direct application of the sufficient conditions from the lectures, and it gives us confidence that the solutions are meaningful.

4. The TPBVP and Shooting: Where the Course Meets Computation

The Euler–Lagrange conditions convert our optimal control problem into a *two-point boundary value problem* (TPBVP): the state x is known at t_0, the costate λ is determined at t_f by transversality, and the two are coupled through the 12D state–costate ODE system. The lectures (pp. 147–148, 186–196) emphasize this structure and present MATLAB's `bvp4c` as the standard numerical tool.

Our project implements the same idea as a *shooting method*: guess the initial costates $\lambda(t_0)$, propagate the full system forward, measure the terminal miss, and use Newton iteration (`scipy.optimize.fsolve`) to drive the miss to zero. This is essentially what `bvp4c` does internally, and it is the approach the course demonstrates on the stirred-tank reactor example (pp. 186–196).

We implement three shooting variants, one for each cost type:

- `solve_min_energy`: 4 unknown costates, 4 terminal residuals (two-body) or 6+6 for CR3BP.
- `solve_min_time`: costates + free t_f; one extra residual from $\mathcal{H}(t_f) = 0$.
- `solve_min_fuel`: costates with homotopy continuation on the smoothing parameter ρ.

This is where the course material is at its most directly applicable. The shooting method *is* the numerical tool the course teaches for optimal control problems.

5. Where We Extend Beyond the Course

The indirect shooting method works beautifully on the textbook examples. On our problem, it runs into two difficulties that the course warns about but does not fully resolve: *sensitivity to initial guesses* and *discontinuous control profiles*. These difficulties motivated everything in this section—techniques that extend the course material rather than directly applying it.

5.1 The Initial Guess Problem

The lectures note (p. 196, summary) that "a good initial guess is important to get an accurate solution, if a solution exists" and that "discontinuous points in the state/costate equations or control function could cause `bvp4c` to fail." In the stirred-tank example, one can guess $\lambda(t_0) \approx [0.5, 0.5]$ and the shooting converges. In the CR3BP, the costate space is 6-dimensional, the dynamics are chaotic, and the basin of convergence for the shooting method is vanishingly small. A random guess for $\lambda(t_0)$ almost never works.

This is the central practical challenge of indirect methods, and it is the reason our project develops an alternative approach.

5.2 Direct Collocation: Discretize First, Optimize Second

The course teaches the *indirect* pathway: derive the continuous necessary conditions (Euler–Lagrange), then discretize and solve the resulting TPBVP. Direct collocation reverses the order: discretize the original optimal control problem into a finite-dimensional nonlinear program (NLP), and let the NLP solver handle the optimality conditions internally.

The theoretical license for this comes from the *constrained optimization* material in the first part of the course (pp. 133–140). The augmented cost $J_a = J + \lambda^\top f(x, u)$ with Lagrange multipliers is exactly what an NLP solver constructs when it forms the KKT system. The KKT multipliers on the dynamics defect constraints are the discrete analogues of the costates $\lambda(t)$. So the same variational structure from the lectures is still present—it is just handled numerically rather than analytically.

Our implementation parameterizes the state trajectory with Bézier curves (Bernstein polynomial basis) and enforces dynamics as equality constraints at Gauss–Legendre collocation points:

$$\text{NLP:} \qquad \min_{P,\,u} \sum_k w_k \, \|u_k\|^2 \, \Delta t \qquad \text{s.t.} \quad \dot{x}_{\text{Béz}}(\tau_k) = f\big(x(\tau_k),\, u_k\big), \quad x(t_0) = x_0, \;\; x(t_f) = x_f.$$

The Bézier parameterization itself is not from the course—it is our contribution. But the problem it solves, the constraints it enforces, and the optimality conditions the solver satisfies are all grounded in the variational framework from the lectures.

5.3 Mesh Refinement: A Computational Strategy for Robustness

The dynamic programming principle from the course (pp. 197–206) teaches that an optimal sub-trajectory of an optimal trajectory is itself optimal. We exploit a computational analogue of this idea through *mesh refinement*: solve a coarse version of the problem first (few segments, loose tolerance), then use that solution to warm-start a finer discretization. Each level inherits a good initial guess from the previous one, sidestepping the initial-guess sensitivity that plagues both shooting and direct methods on hard problems.

The cascade proceeds as:

$$20 \text{ segments} \xrightarrow{\text{warm-start}} 40 \text{ segments} \xrightarrow{\text{warm-start}} 60 \text{ segments}$$

This is not taught in AAE 568, but it is a direct response to the problem the course identifies: that numerical TPBVP solvers need good starting points.

5.4 Homotopy Continuation for Minimum Fuel

The minimum-fuel problem has a bang-bang control structure: thrust is either at full magnitude or zero, determined by the switching function $S = \|\lambda_v\| - 1$. The discontinuity in $u(t)$ is exactly the kind of difficulty the course warns about. We address it with a smoothing technique:

$$T = \frac{u_{\max}}{2}\big(1 + \tanh(S/\rho)\big),$$

where ρ is a smoothing parameter. We start with $\rho = 1$ (nearly linear) and progressively reduce it through a schedule $\rho = 1 \to 0.5 \to 0.1 \to 0.05 \to 0.01 \to 0.001$, using the converged solution from

each stage as the initial guess for the next. Each step makes the problem slightly harder but starts from a nearby solution, and in the limit the smooth approximation recovers the true bang-bang control.

This homotopy continuation is not covered in the course, but the *need* for it arises directly from the inequality-constrained optimal control theory taught in the lectures (pp. 161–166). The switching function, the bang-bang structure, and the complementarity conditions are all from the course; the smoothing and continuation are our engineering response to making them numerically tractable.

5.5 Bridging Indirect and Direct: Warm-Starting Across Methods

Perhaps the most satisfying connection in the project is using the direct collocation solution to warm-start the indirect shooting. The stationarity condition $u^* = -\frac{1}{2}\lambda_v$ gives us a bridge: once IPOPT finds an optimal control profile $u^*(t)$, we can estimate the initial costates as $\lambda_v(t_0) \approx -2\,u^*(t_0)$ and use that to seed the shooting method. This allows us to obtain the indirect solution (which is more analytically interpretable) even when a cold-start shooting would diverge.

This technique is not from the course, but it depends entirely on the course material: without the stationarity condition from the Euler–Lagrange theory, we would have no way to translate between the direct and indirect worlds.

6. Summary

The following table separates what comes directly from AAE 568 from what we built on top of it:

Concept	Lecture Ref.	Role in Project
Directly from the course		
Bolza problem formulation	pp. 141–142	Problem setup (cost + dynamics + BCs)
Hamiltonian & augmented cost	pp. 143–145	Costate ODE derivation
Euler–Lagrange equations	pp. 146–147	State–costate system in `dynamics.py`
Stationarity $\partial\mathcal{H}/\partial u = 0$	p. 147	Control law $u^* = -\frac{1}{2}\lambda_v$
Free-time transversality $\mathcal{H} = 0$	pp. 150–152	Extra residual in min-time shooting
Pontryagin's Minimum Principle	pp. 153–160	Bang-bang thrust for bounded control
Inequality constraints / switching fn.	pp. 161–166	Min-fuel switching $S = \|\lambda_v\| - 1$
TPBVP and shooting method	pp. 186–196	Indirect solver via `fsolve`
Quadratic performance index	p. 142	$\int \|u\|^2 dt$ objective
Sufficient cond. (Legendre–Clebsch)	p. 152	$\mathcal{H}_{uu} = 2I > 0$ verified
Extensions beyond the course		
Direct collocation as NLP	inspired by pp. 133–140	Bézier/IPOPT solver; KKT mult. as discrete costates
Mesh refinement cascade	motivated by p. 196	Coarse-to-fine warm-starting for robustness
Homotopy continuation	motivated by pp. 161–166	tanh smoothing with ρ-schedule for min-fuel
Indirect–direct bridging	uses $\partial\mathcal{H}/\partial u = 0$	Estimate $\lambda(t_0)$ from IPOPT's u^*
CR3BP dynamics & gravity Hessian	—	6D nonlinear dynamics with two singularities
CasADi automatic differentiation	—	Exact sparse Jacobians for NLP

7. Conclusion

This project is, at its foundation, an exercise in AAE 568. The Bolza formulation tells us how to pose the problem. The Hamiltonian and Euler–Lagrange equations give us the necessary conditions. Pontryagin's Principle handles the bounded-thrust case. The shooting method solves the resulting TPBVP. Every one of these steps is a direct application of course material, carried out on equations we wrote by following the derivations in the lecture notes.

Where the project goes beyond the course is in the *engineering* required to make these ideas work on a problem that is harder than the classroom examples: a direct collocation method that sidesteps the initial-guess sensitivity of shooting, a mesh refinement strategy that builds solutions incrementally, and a homotopy scheme that smooths away the discontinuities in bang-bang control. But these extensions are not independent inventions—they are responses to specific limitations that the course itself identifies, built on top of the theoretical tools the course provides. The indirect and direct methods in our code are two sides of the same coin, and that coin was minted in AAE 568.